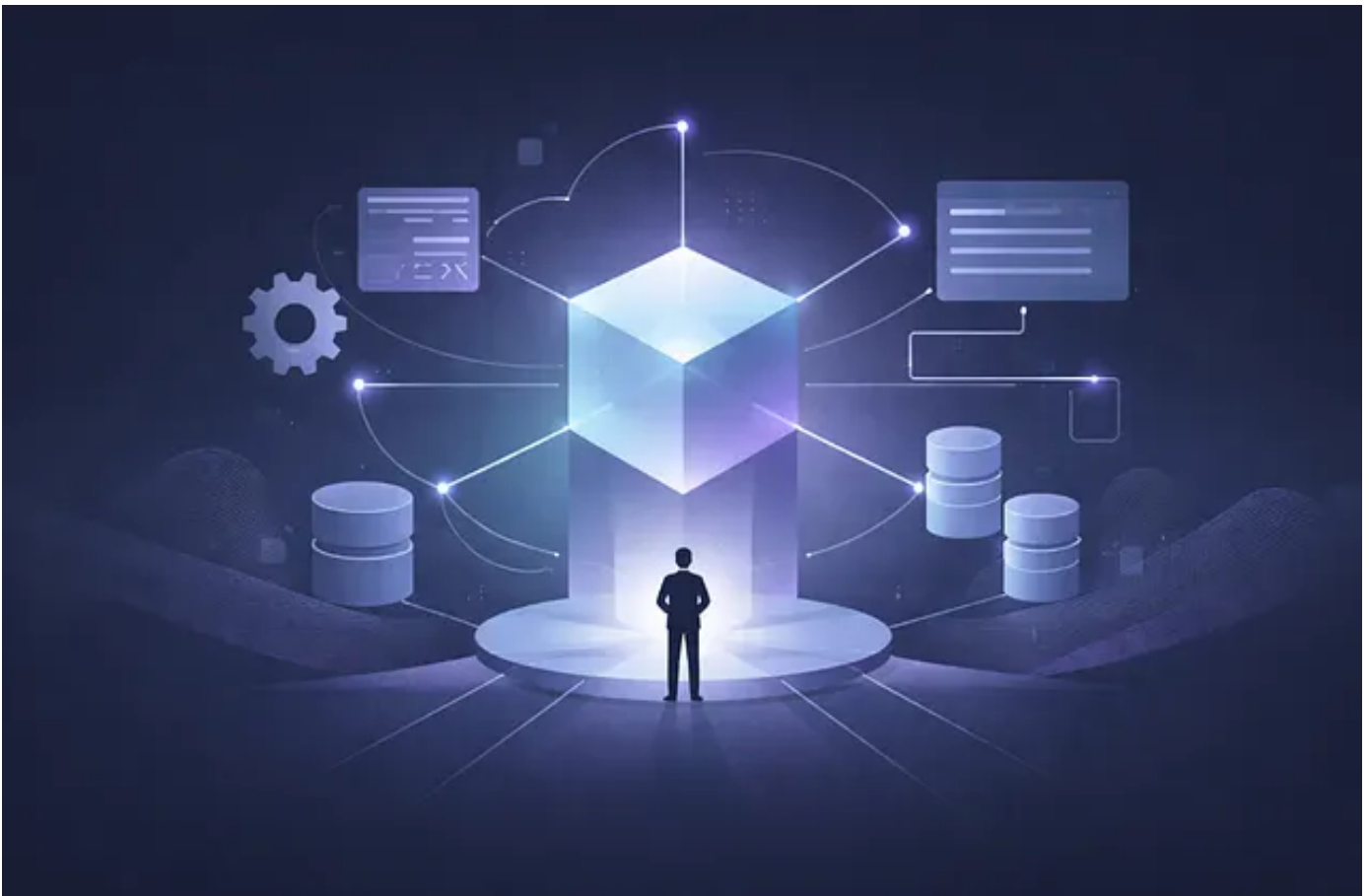


C# MCP SDK Practical Tutorial Developers Love

Master MCP: Real-World C# Integration Guide

[Nagaraj](#)

MCP, C#



Source : Author

The hands-on C# MCP SDK tutorial provides you with tools to improve your work efficiency. The course

provides practical developer-tested code which teaches students to create and test secure MCP servers.

I have developed an extensive guide which shows the process of constructing an MCP server through C# programming. This guide uses both a tool and a prompt which are based on the official SDK.

The guide contains:

- An illustrative one with a `summarize_uri` tool so that web content may be downloaded and summarized.
- Summarization **prompts creation template**, `summarize_message`.
- Creation of a project integrated with VS code, step by step
- **Enhanced version** with statistics and better formatting
- Error handling, async operation, and schema creation **best practices**

The sample copy is MCP SDK ready for production and maintains the conventions in the official C# SDK repository — it is just a matter of copying on to get a functional MCP server up and running in minutes!

Prerequisites

👉 .NET 8.0 SDK or above

👉 In view that they are essential for understanding some examples, it would be better to know C#-async-await basics.

Step 1: Create new console application

Step 2: Adding a NuGet Package

Step 3 : Creating the MCP Server Implementation

Class for Tool input schema for URL summarization

SummarizeUrlInput.cs

Arguments schema prompt

Summarize content from url class (Tool) :

Summarize content (prompt) :

Program.cs :

Step 4: Understanding the Code

Tool Registration (`summarize_uri`)

The tool:

- **Name:** `summarize_uri`
- **Purpose:** Downloads content from a URI and returns a summary
- **Input:** The system requires a URI to be provided as a string
- **Output:** The system produces summary text which includes character count information

Prompt Registration (`summarize_message`)

The prompt:

- **Name:** `summarize_message`
- **Purpose:** Creates an approximate prompting interface to facilitate message summarization

- **Arguments:** You have been requested to paraphrase the message text.
- **Output:** Structured prompt for user message

Step 5: Build and Run

```
dotnet build  
dotnet run
```

The server will start and communicate via stdio (standard input/output).

Step 6: Configure in Claude Desktop

You need to edit the Claude Desktop configuration file according to your operating system:

macOS: ~/Library/Application

Support/Claude/claude_desktop_config.json

Windows: %APPDATA%\Claude\claude_desktop_config.json

Kindly insert this configuration:

Step 7: Test the Server

When Claude Desktop is restarted, the following can be done:

Test the Tool

Ask Claude: "Use the `summarize_uri` tool to get content from <https://modelcontextprotocol.io/docs/getting-started/intro.md>"

Test the Prompt

Ask Claude: Utilize the `summarize_message` prompt to summarize the following long description: [Your long message]

Advanced Example: Likely Usefulness

Following is a more reformed version with word count and better formatting.

Central takeaways

- **McpServer** serves as the fundamental framework which developers use to construct MCP servers.
- The **tool** provides users with functions to execute specific tasks which include content downloading.
- **Prompt**: Tools to prompts for templates-based generation
- **StdioTransport**: The layer for communication with standard I/O
- The **JSON Schema** so defined specifies the structure for

the input and output of tools and user-entry prompts.

Commonly Seen Patterns

Error Handling

All handlers of utility code should always enclose the functionality within a try-catch block and then produce an error response.

Add multiple tools during setting:

Asynchronous Operations

All handlers have support for `async/await` for I/O operations like HTTP requests.

Troubleshooting

The server does not appear on Claude Desktop:

- Inspect the JSON syntax of the configuration file JSON.
- The project path needs to be verified as an absolute path.
- The entire Claude Desktop application needs to be restarted.

The tool execution process has encountered an error:

- The server logs should be examined through the Claude

Desktop developer tools.

- The system requires verification that the input schema matches the expected format.
- The system requires separate testing for each HTTP endpoint.

This example lays a foundation for building more advanced MCP servers using C#.

Thank you for reading!



Hit the applause button and show your love ❤️, and please follow ➡️ for a lot more similar content! Let's keep the good vibes flowing!